```
app.Run();
```

❶ Adds the problem details service to the DI container for use by the ExceptionHandlerMiddleware

❷ When not in development, the pipeline uses ExceptionHandlerMiddleware.

NOTE In listing 10.15 you added the Problem Details services to the DI container so that the `ExceptionHandlerMiddleware` can generate a Problem Details response automatically. As you're adding the extra middleware only in Staging and Production, you could add the services conditionally to the DI container too instead of always adding them as we did here.

You can inject `IHostEnvironment` anywhere in your app, but I advise against using it in your own services outside Program.cs. It's far better to use the configuration providers to customize strongly typed settings based on the current hosting environment and inject these settings into your application instead.

As useful as it is, setting `IHostEnvironment` with an environment variable can be a little cumbersome if you want to switch back and forth among different environments during testing. Personally, I'm always forgetting how to set environment variables on the various operating systems I use. The final skill I'd like to teach you is how to set the hosting environment when you're developing locally.

10.4.3 Setting the hosting environment

In this section I show you a couple of ways to set the hosting environment when you're developing. These techniques make it easy to test a specific app's behavior in different environments without having to change the environment for all the apps on your machine.

If your ASP.NET Core application can't find an `ASPNETCORE_ENVIRONMENT` environment variable when it starts up, it defaults to a production environment, as shown in figure 10.6. So when you deploy to production, you'll be using the correct environment by default.